

Decision Trees

▷ A very simple architecture for classification tasks, approximating functions, etc.

- Fast! Sometimes simple is all you need.

Fast to train. Fast to use $\sim \log(\text{size of training data})$.

- No learned parameters

Rather — minimization is done exactly, not with gradient descent. Interpretable!

- A few tunable "hyperparameters," e.g. depth.

- Takes all different sorts of data: numbers, tokens (labels)

and no need to normalize/pre-process.

Decision Trees

▷ A very simple architecture for classification tasks, approximating functions, etc.

- Fast! Sometimes simple is all you need.

Fast to train. Fast to use $\sim \log(\text{size of training data})$.

- No learned parameters

Rather — minimization is done exactly, not with gradient descent. Interpretable!

- A few tunable "hyperparameters," e.g. depth.

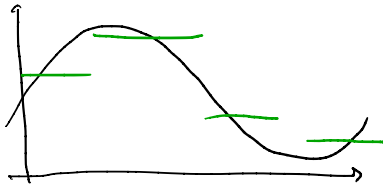
- Takes all different sorts of data: numbers, tokens (labels)

and no need to normalize/pre-process.

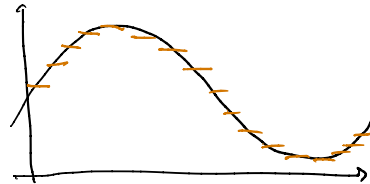
▷ The sort of functions it approximates with are piecewise linear

Higher depth $\leftrightarrow$ more linear segments.

low depth



high depth



$\leadsto$ very high depth can be perfect (or overfitted...)

Decision Trees

Example: Suppose you're given a representative basket of fruit (training data)

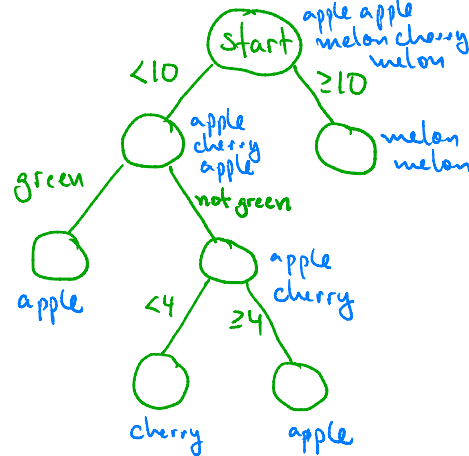
diameter	color	label
7cm	red	apple
6	green	apple
20	green	melon
15	yellow	melon
2	red	cherry

Decision Trees

Example: Suppose you're given a representative basket of fruit (training data)

diameter	color	label
7cm	red	apple
6	green	apple
20	green	melon
15	yellow	melon
2	red	cherry

Here's a decision tree:

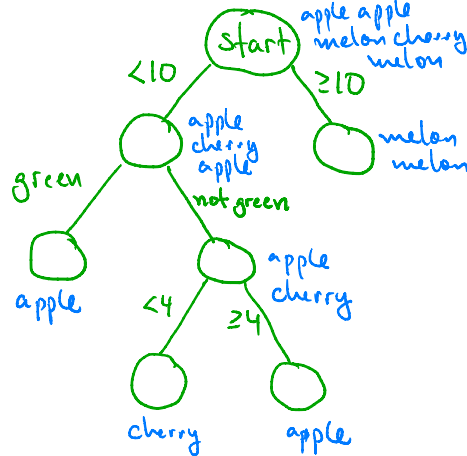


Decision Trees

Example: Suppose you're given a representative basket of fruit (training data)

diameter	color	label
7cm	red	apple
6	green	apple
20	green	melon
15	yellow	melon
2	red	cherry

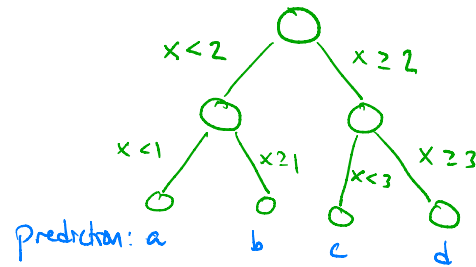
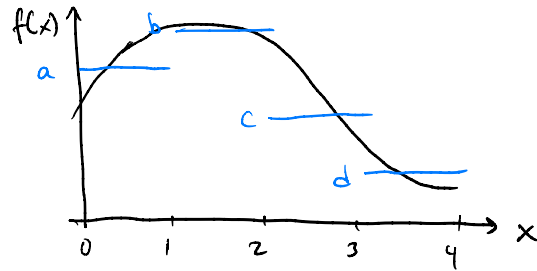
Here's a decision tree:



- could now use it to try to classify any fruit
- could have stopped at an intermediate depth and gotten a probability estimate instead of a perfect classification — often useful, as the perfect classification takes more time and can be overfitted.

Decision Trees

For functions it's similar



@ depth 2

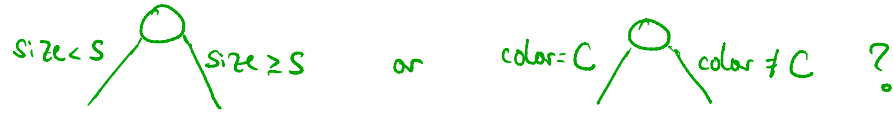
Training

Want to optimize the classification task (or regression for functions).

Training

Want to optimize the classification task (or regression for functions).

To create an optimal tree, start at the top

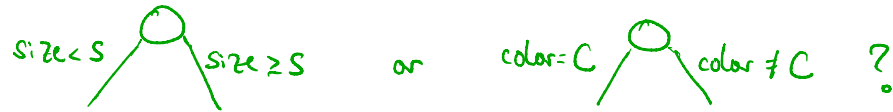


Choose the feature and threshold that minimizes the "impurity" of the branches.

Training

Want to optimize the classification task (or regression for functions).

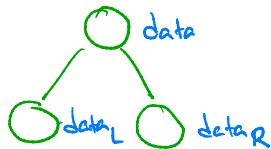
To create an optimal tree, start at the top



Choose the feature and threshold that minimizes the "impurity" of the branches.

Define an impurity function $H: \{\text{set of labelled data}\} \rightarrow \mathbb{R}_{\geq 0}$ or $[0, 1]$

For

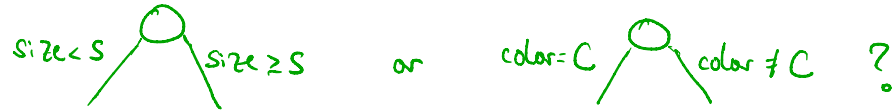


minimize $\frac{|data_L|}{|data|} H(data_L) + \frac{|data_R|}{|data|} H(data_R)$

Training

Want to optimize the classification task (or regression for functions).

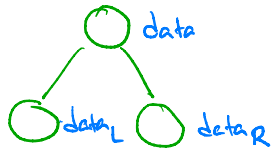
To create an optimal tree, start at the top



Choose the feature and threshold that minimizes the "impurity" of the branches.

Define an impurity function $H: \{\text{set of labelled data}\} \rightarrow \mathbb{R}_{\geq 0}$ or $[0, 1]$

For



minimize $\frac{|data_L|}{|data|} H(data_L) + \frac{|data_R|}{|data|} H(data_R)$

Common choices: Entropy $H(data) = - \sum_{\text{classes } i} p_i \log p_i$

Gini impurity $H(data) = \sum_{\text{classes } i} p_i (1 - p_i)$ ← probability of mis-classifying a randomly chosen element

For function approximation, $H = \text{Mean Square Error}$

Random Forests

"Have a decision to make? Poll a large group of friends and go with the average response."

↑
each of which might only know about part of your concerns

Used when training data is large and/or its features are complex.

Random Forests

"Have a decision to make? Poll a large group of friends and go with the average response."

↑
each of which might only know about part of your concerns

Used when training data is large and/or its features are complex.

A Random Forest Classifier first takes N ^{hyperparameter} random samples from the data.

Then trains N separate decision trees on these samples.

And finally, averages the results of the N trees to produce a final classification.

Random Forests

"Have a decision to make? Poll a large group of friends and go with the average response."

↑
each of which might only know about part of your concerns

Used when training data is large and/or its features are complex.

A Random Forest Classifier first takes N ^{hyperparameter} random samples from the data.

Then trains N separate decision trees on these samples.

And finally, averages the results of the N trees to produce a final classification.

Example: for classifying image (or written digits...), a RFC might have decision trees that each see a small fraction of the pixels — part of the image.

Random Forests

"Have a decision to make? Poll a large group of friends and go with the average response."

↑
each of which might only know about part of your concerns

Used when training data is large and/or its features are complex.

A Random Forest Classifier first takes N ^{hyperparameter} random samples from the data.

Then trains N separate decision trees on these samples.

And finally, averages the results of the N trees to produce a final classification.

Example: for classifying image (or written digits...), a RFC might have decision trees that each see a small fraction of the pixels — part of the image.

(*) RFC's come with a way to test their internal consistency.

Since each training datapoint is invisible ("out-of-bag") to some of the N trees during training, can check if these trees predict its label correctly. $\leadsto$ out-of-bag accuracy.